

路网图建模与二维平面图建模下 P2-P4 实现方案对比

本文比较两种仓储 AMR 调度建模方案：

方案 A：仓库路网图模型

仓库 = 节点 + 通道边

方案 B：二维平面地图模型

仓库 = 平面区域 + 障碍物 + 机器人自由规划路径

核心区别是：

- P2 主要依赖路径成本矩阵，因此两种方案差别不大；
- P3 和 P4 需要处理时空占用、冲突检测和动态重排，因此二维平面图方案会明显复杂很多。

1. 总体对比

模块	路网图方案	二维平面图方案
P1 输出	节点对之间的路径、边序列、成本	点对之间的连续路径折线、轨迹采样、成本
P2 任务分配	直接使用成本矩阵	仍然使用成本矩阵，变化不大
P2 任务排序	使用任务点之间的路网路径成本	使用任务点之间的连续路径成本
P3 时间调度	展开 edge / node 占用	展开连续轨迹 (x, y, t)
P3 冲突检测	同时占用同一边或节点即冲突	两机器人距离小于安全距离即冲突
P4 动态重排	封锁边、调整边权、局部重算	封锁区域、多边形避障、连续空间重规划
实现难度	中等，适合运筹学大作业	中高，容易变成机器人路径规划项目
报告解释	清楚，约束容易表达	更真实，但模型和代码复杂度更高

2. P2：任务分配与任务排序

P2 要回答：

哪台 AMR 做哪个任务？

每台 AMR 按什么顺序做任务？

P2 本质上需要的是路径成本：

AMR 初始点 -> 任务取货点

任务取货点 -> 任务送货点

上一个任务送货点 -> 下一个任务取货点

因此，P2 并不太关心底层是路网图还是二维平面图。只要 P1 能提供统一接口，P2 就能继续工作。

统一接口可以是：

```
from, to, path_uid, travel_time, distance, total_cost
```

2.1 路网图方案下的 P2

P1 输出类似：

```
from_node, to_node, path_uid, travel_time, distance, total_cost, edge_sequence
```

任务 τ_j 分配给 AMR k 的成本可以写成：

```
Cost(k, j)
= cost(init_k, pickup_j)
+ cost(pickup_j, delivery_j)
```

任务排序时，如果 AMR 做完 τ_1 后接着做 τ_2 ，则衔接成本为：

```
cost(T1.delivery, T2.pickup)
```

优点：

- 成本查询简单；
- 可以预先生成完整成本矩阵；
- 容易写成 0-1 指派模型、整数规划或启发式算法；
- 后续 P3 可以直接知道路径经过哪些通道。

缺点：

- 路径受预定义路网限制；
- 不如二维自由空间路径真实。

2.2 二维平面图方案下的 P2

P1 输出可能变成：

```
from_point, to_point, path_uid, travel_time, distance, total_cost, path_geometry
```

其中 `path_geometry` 是路径折线，例如：

```
[(0,1), (2,1), (4,3), (6,3)]
```

P2 仍然可以使用相同的成本逻辑：

```
Cost(k, j)
= planner(init_k, pickup_j).total_cost
+ planner(pickup_j, delivery_j).total_cost
```

但二维路径规划计算更贵，不能在 P2 排序过程中频繁临时调用路径规划器。因此即使使用二维平面图，仍然建议 P1 先生成缓存表：

```
path_cost.csv
```

优点：

- 成本更接近真实自由空间路径；
- 可以绕开任意形状障碍物；
- 路径展示效果更真实。

缺点：

- P1 的路径生成更复杂；
- 如果任务点很多，任意两点路径规划量会增加；
- 后续 P3 不能再依赖 `edge_sequence` 简单做冲突检测。

2.3 P2 小结

P2 在两种方案下变化最小。

可以理解为：

P2 不关心路径是怎么来的，只关心路径成本是多少。

因此，只要 P1 保证输出统一的 `path_cost.csv`，P2 基本不用大改。

3. P3：时间调度、路径占用与冲突检测

P3 要回答：

每个任务什么时候开始？

每个任务什么时候完成？

AMR 在什么时间占用哪里？

是否发生冲突？

这里两种方案差别最大。

3.1 路网图方案下的 P3

路网方案中，路径占用是离散的。

例如：

```
path = IN1 -> J1 -> P1
edge_sequence = e01 -> e05
```

如果 AMR 在 $t=0$ 出发：

时间段	占用对象
0-2	e01
2-4	e05

通道冲突检测规则：

同一时间段内，占用同一 edge 的 AMR 数 > edge.capacity
=> 通道冲突

路口冲突检测规则：

同一时刻，到达同一 node 的 AMR 数 > node.capacity
=> 路口冲突

对应输出：

schedule_result.csv
edge_occupancy_schedule.csv
node_occupancy_schedule.csv
conflict_log.csv

优点：

- 冲突检测容易实现；
- 通道容量约束、路口容量约束都很清楚；
- 适合写成运筹模型中的网络容量约束；
- 报告里容易解释；
- 与仓库通道管理逻辑一致。

缺点：

- 冲突判断较粗糙；
- 两台车在不同边但空间距离很近时，路网模型可能无法识别；
- 真实机器人轨迹被简化。

3.2 二维平面图方案下的 P3

二维方案下，P3 不能只看 `edge_id`，因为路径是连续折线。

P1 或 P3 需要生成轨迹采样表：

`trajectory_sample.csv`

示例：

path_uid	time_offset	x	y	heading
P001	0.0	0.0	1.0	0
P001	0.5	0.5	1.0	0
P001	1.0	1.0	1.0	0

P3 根据任务开始时间，把相对时间转成绝对时间：

`absolute_time = task_start_time + time_offset`

冲突检测变成：

对任意两台 AMR，在同一时刻 `t`：
`distance((x1,y1), (x2,y2)) < safety_distance`
=> 冲突

更精确时，还要检查机器人扫掠区域是否相交：

两台 AMR 的扫掠圆盘或矩形在时间区间内相交
=> 冲突

优点：

- 更接近真实机器人运动；
- 可以检测非路网边上的空间距离冲突；
- 轨迹动画展示更自然；
- 可以处理任意障碍物和自由空间路径。

缺点：

- 需要时间离散化；
- 采样太粗会漏检冲突；
- 采样太细会增加计算量；
- 冲突修复不再只是“某条边等待”，而是要调整连续轨迹；
- 运筹模型约束更难写。

3.3 P3 小结

路网图方案下，P3 是：

边 / 节点占用调度

二维平面图方案下，P3 是：

连续轨迹时空碰撞检测

后者已经明显从运筹调度问题转向机器人运动规划问题。

4. P4：动态扰动与重排

P4 要回答：

通道封锁、机器人延误、新任务出现后，原计划如何调整？

这里两种方案差别也很明显。

4.1 路网图方案下的 P4

动态事件可以定义成：

edge_block: e10 在 18-24 封锁

amr_delay: AMR2 在 24-30 延误

new_task: t=26 出现 T13

处理逻辑：

1. 读取 `edge_occupancy_schedule.csv` ；
2. 找出哪些任务在封锁时间窗内经过 `e10` ；
3. 标记受影响任务；
4. 选择处理方式：
 - 等待；
 - 换候选路径；
 - 重排任务顺序；
 - 局部重算未来任务；
5. 输出 `reschedule_result.csv` 。

通道封锁处理非常自然：

删除边 `e10`

或把 `e10` 的 `cost` 设置为无穷大

重新计算受影响 `from_node -> to_node` 的候选路径

优点：

- 受影响任务容易识别；
- 封锁事件容易表达；
- 局部重排逻辑清楚；
- 指标容易统计，例如新增延期、扰动任务数、重排时间；
- 适合报告中的“通道封锁与动态重排”实验。

缺点：

- 动态事件主要局限于节点或边；
- 不容易表达自由空间里的临时障碍区域。

4.2 二维平面图方案下的 P4

二维方案下，动态事件不再是封锁边，而是封锁区域：

```
blocked_region = rectangle / polygon  
time_window = 18-24
```


P4 要做：

1. 判断哪些路径几何与封锁区域相交；
2. 判断相交时间是否落在封锁时间窗；
3. 对受影响任务重新调用 P1 路径规划器；
4. 生成新几何路径；
5. 重新采样轨迹；
6. 重新做 P3 的连续空间冲突检测；
7. 如果还有冲突，继续等待或重规划。

受影响判断变成：

```
path_geometry intersects blocked_polygon  
且  
path_time overlaps event_time_window
```

优点：

- 动态事件表达更真实；
- 可以模拟临时货物堆放、人员占用区域、地面障碍物；
- 展示效果更强。

缺点：

- 需要几何库或自己写相交检测；
- 需要重新规划连续路径；
- 需要重新生成轨迹采样；
- 冲突检测和重排可能反复迭代；
- 很容易超出运筹学大作业范围。

4.3 P4 小结

路网图方案下，P4 是：

边封锁 + 局部重排

二维平面图方案下，P4 是：

区域封锁 + 连续空间重规划 + 轨迹重采样

后者明显更复杂。

5. 数据接口对比

5.1 路网图方案推荐接口

path_cost.csv:

from_node, to_node, path_uid, travel_time, distance, total_cost, edge_sequence

path_edge_occupancy.csv:

path_uid, edge_id, offset_start, offset_end, capacity

path_node_occupancy.csv:

path_uid, node_id, offset_time

P2 主要读取:

path_cost.csv

P3/P4 主要读取:

path_edge_occupancy.csv

path_node_occupancy.csv

5.2 二维平面图方案推荐接口

path_cost.csv:

from_point, to_point, path_uid, travel_time, distance, total_cost, path_geometry

trajectory_sample.csv:

path_uid, time_offset, x, y, heading, speed

spatial_occupancy.csv:
path_uid, time_offset, x, y, safety_radius

P2 主要读取：

path_cost.csv

P3/P4 主要读取：

trajectory_sample.csv
spatial_occupancy.csv

6. 实现难度对比

项目	路网图方案	二维平面图方案
P1 路径生成	中等	较难
P2 任务分配	中等	中等
P3 时间调度	中等	较难
P3 冲突检测	中等偏简单	较难
P4 动态重排	中等	很难
报告解释	清楚	需要更多机器人规划背景
适合运筹学	很适合	容易跑偏
展示效果	清楚但工程感一般	很强
项目风险	可控	较高

7. 推荐折中方案

最稳的方案不是完全二选一，而是：

外观上使用二维仓库平面图
内部仍然保留图论接口

也就是：

二维仓库平面图 / 仓库示意图



人工或自动生成路网图



P1 计算候选路径与成本



P2 任务分配



P3 时间调度与冲突检测



P4 动态重排

这样既可以展示仓库空间布局，又不会把后续模块全部拖进连续空间机器人运动规划。

报告中可以这样表述：

真实仓储环境虽然是二维空间，但 AMR 通常沿预定义通道、虚拟轨道或导航拓扑运行。因此，本文将仓库可通行区域抽象为图论网络，在保留通道结构和关键节点的同时，使路径成本矩阵、任务分配、时间调度和冲突检测能够统一建模。

8. 最终建议

如果目标是运筹学大作业，建议主线采用：

方案 A：路网图方案

理由：

- 更符合运筹学中的网络建模；
- P2-P4 接口清晰；
- 冲突检测和动态重排容易表达为容量约束和时间窗约束；
- 工作量可控；
- 报告中可以清楚解释模型、变量、约束和指标。

如果想增强展示效果，可以采用折中方式：

二维仓库图用于展示
路网图用于建模和计算

不要直接把 P3/P4 改成连续空间碰撞检测，否则项目会明显偏向机器人路径规划与仿真，工作量和风险都会增大。